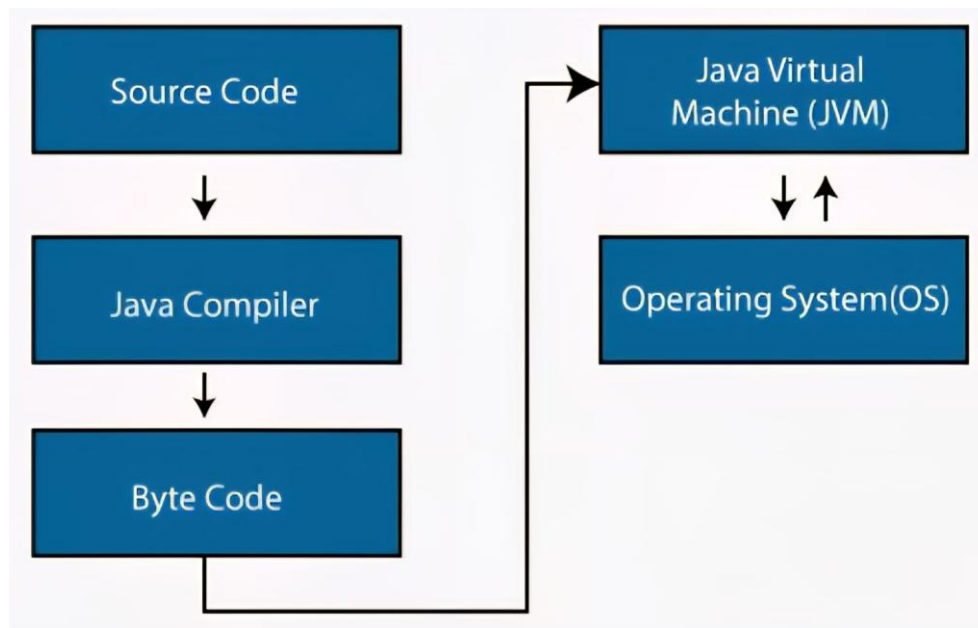


Java

Architecture of Java :-

- In Java, there is a process of compilation and interpretation.
- The code written in Java , is converted into byte codes which is done by the Java Compiler.
- The byte codes, then are converted into machine code by the JVM.
- The Machine code is executed directly by the machine.



Main Components of Java Architecture :-

(a) Java Development Kit (JDK) -

- A software development environment for Java.
- Includes:
 - Compiler (javac) → Converts source code into bytecode.
 - Java Runtime Environment (JRE) → To execute programs.
 - Tools → Debugger, Archiver, Documentation tools.

(b) Java Runtime Environment (JRE) -

- Provides libraries, Java Virtual Machine (JVM), and other files to run programs.
- Does not include compiler.
- Used only to run programs, not to develop.

(c) Java Virtual Machine (JVM)

- The heart of Java architecture.
- Converts bytecode into machine code for execution.
- Platform-dependent (different JVMs for Windows, Linux, Mac), but bytecode is platform-independent.

Difference Between JDK, JRE & JVM -

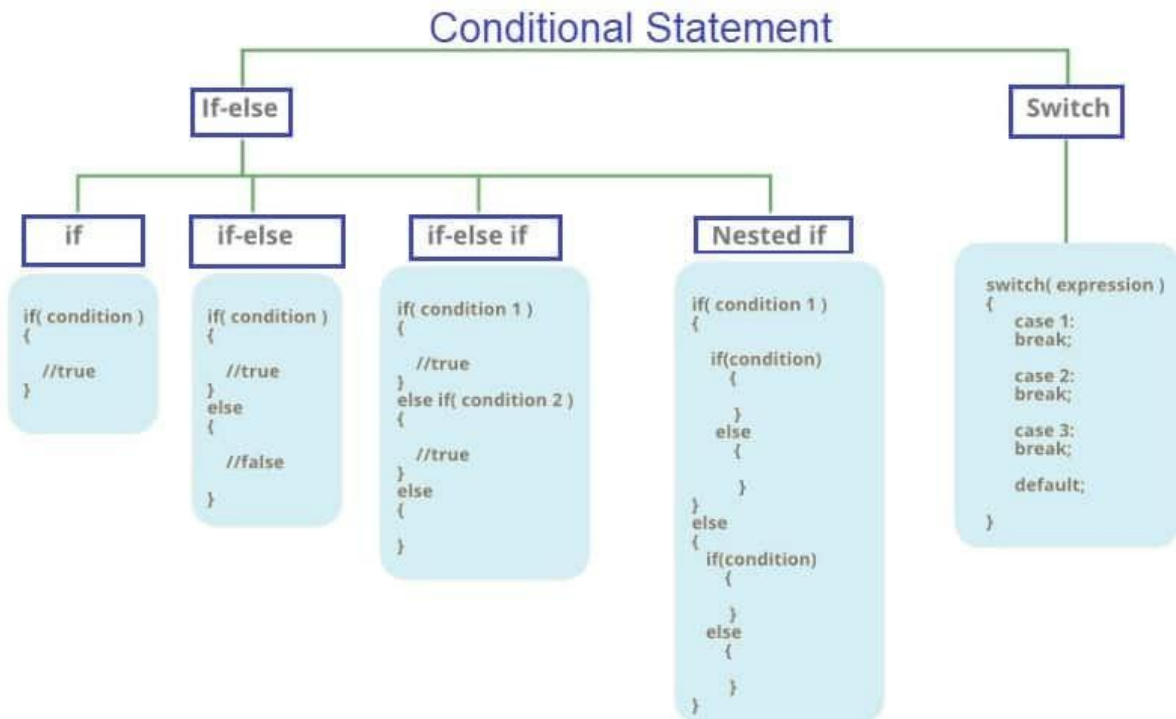
JDK	JRE	JVM
JDK stands for Java Development Kit	JRE stands for Java Runtime Environment	JVM stands for Java Virtual Machine
Java Development kit is used for developing Java applications and applets.	Java Runtime Environment is provided by Java to run the applications.	Java Virtual Machine is a specification that provides a run-time environment for Java applications and describes the requirement of JVM implementation.
It is platform-dependent.	It is platform-dependent.	It is platform-independent.
Implementation is JDK=JRE+Development tools	Implementation is JRE=JVM+libraries	Implementation is only a runtime environment to execute the code.
JDK consists of many development tools.	JRE consists of class libraries and other files.	JVM does not consist of any software tools.

Difference Between JDK 21 & JDK 17 –

Feature	Java 17	Java 21
Sealed Classes	Introduced sealed classes, allowing more control over class inheritance.	No changes; already present in Java 17.
Pattern Matching for Switch	Preview feature for extending pattern matching to switch statements.	Finalized feature, making switch more concise and expressive.
Strong Encapsulation of JDK Internals	Strengthened encapsulation by making internal JDK APIs inaccessible by default.	No changes; already present in Java 17.
UTF-8 by Default	Not available.	UTF-8 becomes the default character set for String, file I/O, and other APIs.
Simple Web Server	Not available.	Introduced a simple web server for prototyping and testing.
Virtual Threads	Not available.	Virtual threads (Preview) simplify the creation of high-throughput concurrent applications.
Structured Concurrency	Not available.	Introduced (Incubator) API to treat multiple tasks running in different threads as a single unit of work.
Record Patterns	Not available.	Record patterns (Preview) simplify destructuring record values directly in patterns.
Scoped Values	Not available.	Scoped values (Incubator) offer an alternative to thread-local variables.
Sequenced Collections	Not available.	Sequenced collections introduced to maintain a defined iteration order in collections.

Conditional Statements in Java :-

- Conditional statements control the flow of program execution.
- They allow decisions to be made based on conditions (true/false).
- Different code blocks execute depending on the condition.



Types of Conditional Statements :-

1. if Statement

- Executes a block of code only if the condition is true.

Syntax:

```
if (condition) {
    // code executes if condition is true
}
```

Example:

```
int age = 20;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

Output –

You are eligible to vote.

2. if-else Statement

- Executes one block if condition is true, otherwise another block.

Syntax:

```
if (condition) {  
    // code if condition is true  
} else {  
    // code if condition is false  
}
```

Example:

```
package Basic_Concept;
```

```
public class ifelse_statement {  
    public static void main(String [] args)  
    {  
        int num=90;  
        if(num!=90){  
            System.out.println("Equals");  
        }  
        else{  
            System.out.println("Not Equals");  
        }  
    }  
}
```

Output –

Not Equals

3. if-else-if Ladder

- Checks multiple conditions one by one.
- The first true condition's block will execute.

Syntax:

```
if (condition1) {  
    // code  
} else if (condition2) {  
    // code  
} else {  
    // code  
}
```

Example:

```
package Basic_Concept;
```

```
public class elseif_ladder {  
    public static void main(String [] args)  
    {  
        int num=10;  
        if(num==20)  
        {  
            System.out.println("Condition 1 is true");  
        }  
        else if(num>=55){  
            System.out.println("Condition 2 is true");  
        }  
        else if(num<9){  
            System.out.println("Condition 3 is true");  
        }  
        else{
```

```
        System.out.println("All Conditions are False");
    }
}
}
```

Output –

All Conditions are False

4. Nested if Statement

- An if statement inside another if.

Syntax:

```
if (condition1) {  
    // code if condition1 is true  
  
    if (condition2) {  
        // code if both condition1 and condition2 are true  
    }  
}
```

Example:

```
package Basic_Concept;  
  
public class nestedifelse {  
    public static void main(String [] args) {  
        boolean iselectricity=true;  
        boolean isswitchon=true;  
  
        if(iselectricity==true)  
        {  
            if(isswitchon==true)  
            {  
                System.out.println("Ligths on!");  
            }  
            else{  
                System.out.println("Lights off!");  
            }  
        }  
    }  
}
```

```
        System.out.println("No Electricity");
    }
}
}
```

Output –

Lights on!